2. NumPy

NumPy je biblioteka otvorenog koda za Python izuzetno pogodna za probleme linearne algebre, brze i efikasne operacije nad matricama ili velikim nizovima a koristi se kao temelj za mnoge biblioteke u PyData ekosistemu - zbiru biblioteka i okvira, primarno u Python-u, koji za fokus ima nauku o podacima.

Više o biblioteci se može saznati na zvaničnom sajtu koji sadrži dokumentaciju NumPy biblioteke: https://numpy.org/doc/stable/
Počnimo sa komandom za uvoz NumPy biblioteke koja nam omogućava da njene funkcionalnosti koristimo u daljem kodu.

In [2]:
```
import numpy as np
```

Samu biblioteku smo "imenovali" sa standardnom skraćenicom np koja će nam pozivanje biblioteke olakšati u daljem kodu.

Sekcija 1 - NumPy funkcionalnosti, NumPy nizovi Jedan od gradivnih elemenata biblioteke su nizovi i matrice, fokusirajmo se u prvom delu na nizove i definišimo jedan niz tako što ćemo običnu Python listu konvertovati u NumPy niz.

In [3]:
```
lista = [1,2,3]
niz_np = np.array(lista)
niz_np
```

Out[3]:
```
array([1, 2, 3])
```

Proverimo tim niz_np promenljive:

In [4]:
```
type(niz_np)
```

Out[4]:
```
numpy.ndarray
```

Po tipu niz_np promenljive zaista zaključujemo da smo kreirali jedan NumPy niz. Postoje brojne ugrađene funkcije za kreiranje niza pa ih možemo isprobati. Funkcija arange vraća jednako raspodeljene vrednosti u datom intervalu, sa mogućnošću definisanja koraka. Gornja granica se ne uzima za deo niza.

In [6]:
```
niz2 = np.arange(0,10,2)
niz2
```

Out[6]:
```
array([0, 2, 4, 6, 8])
```

Možemo kreirati niz sačinjen od nula ili jedinica uz pomoć funkcije zeros i ones:

In [7]:
```
np.zeros(5)
```

Out[7]:
```
array([0., 0., 0., 0., 0.])
```

In [8]:
```
np.ones(5)
```

Out[8]:
```
array([1., 1., 1., 1., 1.])
```

Možemo iskoristiti i linspace funkciju koja vraća jednako raspoređene brojeve u specificiranom intervalu - dakle ne zadajemo korak kao treći argument već sam broj elemenata koje želimo da uključimo:

In [12]:
```
np.linspace(0,10,10)
```

Out[12]:
```
array([ 0.        ,  1.11111111,  2.22222222,  3.33333333,  4.44444444,
        5.55555556,  6.66666667,  7.77777778,  8.88888889, 10.        ])
```

Možemo kreirati niz pomoću funkcije rand koja pseudonasumično generiše brojeve koji su zapravo slučajan uzorak uniformne raspodele u intervalu [0,1)

In [13]:
```
np.random.rand(5)
```

Out[13]:
```
array([0.64298373, 0.50223562, 0.38509172, 0.61754146, 0.89430236])
```

Još jedan način za kreiranje np niza je uz pomoć funkcije randn koja za razliku od rand funkcije vraća uzorak iz normalne raspodele.

In [14]:
```
np.random.randn(30)
```

Out[14]:
```
array([ 0.44190614, -1.26312217,  0.89542582, -0.52836417,  0.94806171,
        1.19528149,  1.19546057,  0.56053507, -0.13810981, -1.11488552,
        0.71197048, -0.59254991, -1.31871524, -0.22666973, -1.27352068,
        1.35103397, -1.89291575, -1.21756526,  0.25861132, -1.33244693,
        1.36762306,  0.09897005, -0.37720965, -0.351099  , -0.12750262,
       -0.06450382, -2.35769614,  1.14572558, -0.08949903, -0.6662975 ])
```

A postoji i funkcija koja nam vraća slučajne celobrojne vrednosti:

In [15]:
```
np.random.randint(1,100,10)
```

Out[15]:
```
array([ 8, 50, 85, 84, 59, 75, 56, 91, 83, 58])
```

Sekcija 2 - NumPy funkcionalnosti, NumPy korisne funkcije nad nizovima

Pokažimo neke od korisnih funkcija nad nizovima. Počnimo od funkcija max, min, argmax, argmin. Funkcija max kako joj i naziv kaže nalazi maksimalan element niza, funkcija min nalazi minimum a funkcije argmax i argmin indeks maksimalnog i minimalnog respektivno.

In [82]:
```
niz3 = np.array([10,456,6467,2342356,22,13,6,3,1])
```

In [22]:
```
print(np.max(niz3),np.min(niz3),np.argmax(niz3),np.argmin(niz3))
```

```
2342356 1 3 8
```

shape je atribut koji np nizovi poseduju, sam naziv je veoma intuitivan - atribut nam vraća oblik(dimenzije, tj. dužinu niza jer je on jednodimenzionalan) datog np niza, kasnije jako koristan atribut za matrice - možemo ispitati dimenzije matrice.

In [24]:
```
niz3.shape
```

Out[24]:
```
(9,)
```

Zaista, dobili smo dužinu tj. broj elemenata promenljive niz3, a možemo isti rezultat dobiti i funkcijom len.

len(niz3)

Možemo ispitati tip podatka u nizu preko funkcije dtype.

In [26]:
```
niz3.dtype
```

Out[26]:
```
dtype('int32')
```

Elementi niza mogu biti uklonjeni korišćenjem funkcije delete. Kao parametre funkcije prosleđujemo niz i poziciju indeksa elementa koji želimo da izbacimo iz niza. Rezultat moramo upisati nazad u niz!

In [83]:
```
print(niz3)
niz3 = np.delete(niz3,[-1])
print(niz3)
```

```
[     10     456    6467 2342356      22      13       6       3       1]
[     10     456    6467 2342356      22      13       6       3]
```

Šta ako ipak želimo da vratimo taj poslednji element? Numpy ima tu situaciju rešenu sa funkcijom append.

In [84]:
```
niz3 = np.append(niz3,1)
niz3
```

Out[84]:
```
array([     10,     456,    6467, 2342356,      22,      13,       6,
             3,       1])
```

Postoji i funkcija insert koja će ubaciti element na indeks koji prosledimo kao drugi parametar u funkciju. Prvi parametar je sam niz u koji upisujemo datu vrednost a poslednji,

treći parametar je sama vrednost koju ubacujemo.

In [85]:
```
niz3 = np.insert(niz3,4,222)
```

In [86]:
```
print(niz3)
```

```
[     10     456    6467 2342356     222      22      13       6       3
       1]
```

Potrebno je voditi računa o dužini niza!

In [87]:
```
niz3 = np.insert(niz3,12,800)
```

```
---------------------------------------------------------------------------
IndexError                                Traceback (most recent call last)
Cell In[87], line 1
----> 1 niz3 = np.insert(niz3,12,800)

File ~\anaconda3\Lib\site-packages\numpy\lib\function_base.py:5506, in insert(arr, o
bj, values, axis)
   5504 index = indices.item()
   5505 if index < -N or index > N:
-> 5506     raise IndexError(f"index {obj} is out of bounds for axis {axis} "
   5507                      f"with size {N}")
   5508 if (index < 0):
   5509     index += N

IndexError: index 12 is out of bounds for axis 0 with size 10
```

Sekcija 3 - NumPy funkcionalnosti, NumPy indeksiranje niza

Pokažimo sada koliko je np niz zaista fleksibilna struktura i koliko lako možemo pristupati delovima niza, pojedinačnim elementima, podnizu itd.

Pristupanje pojedinačnom elementu niza je jako nalik pristupanju pojedinačnom elementu liste - preko indeksa. Recimo da želimo da pristupimo prvom elementu i poslednjem elementu.

In [27]:
```
niz3[0]
```

Out[27]: 10

In [28]:
```
niz3[-1]
```

Out[28]: 1

Dakle, važe ista pravila kao i kod liste. Možemo selektovati neki opseg niza, recimo od prvog do petog ili od drugog do petog elementa.

In [29]:
```
niz3[0:5]
```

Out[29]:
```
array([     10,     456,    6467, 2342356,      22])
```

In [30]:
```
niz3[1:5]
```

Out[30]:
```
array([    456,    6467, 2342356,      22])
```

Šta numpy niz razlikuje od obične liste? Postoji jedan od opcija koje np niz ima a obična lista naravno ne i ona se naziva "broadcasting" - davanje vrednosti elementima u nekom opsegu, recimo da ćemo elementima od prvog do trećeg dati vrednost 100:

In [32]:
```
niz3[0:3] = 100
niz3
```

Out[32]:
```
array([    100,     100,     100, 2342356,      22,      13,       6,
              3,       1])
```

Sledeća sekcija koda je izuzetno bitna. Šta se dešava kada napravimo podniz, i promenimo mu vrednosti? Napišimo primer.

In [34]:
```
niz4 = np.arange(0,11)
niz4
```

Out[34]:
```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

In [35]:
```
niz5 = niz4[0:6]
niz5
```

Out[35]:
```
array([0, 1, 2, 3, 4, 5])
```

In [36]:
```
niz5[:] = 123
niz5
```

Out[36]:
```
array([123, 123, 123, 123, 123, 123])
```

In [65]:
```
niz4
```

Out[65]:
```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
```

Vidimo da se i originalni niz promenio, dolazimo do važnog zaključka - numpy nizovi su mutible struktura podataka! Dakle, podaci nisu kopirani već samo postoje dva pokazivača na iste memorijske lokacije. Kako da rešimo ovaj problem - želimo kopiju niza. Numpy to rešava funkcijom copy!

In [39]:
```
niz5_copy = niz5.copy()
niz5_copy[:] = 1
print(niz5_copy)
print(niz5)
```

```
[1 1 1 1 1 1]
[123 123 123 123 123 123]
```

Još jedan od korisnih načina selekcije - selekcija po uslovu. Recimo da želimo sve elemente koji su veći od 5.

In [41]:
```
niz4 = np.arange(0,11) #resetujemo niz
niz4[niz4>5]
```

Out[41]:
```
array([ 6,  7,  8,  9, 10])
```

Možemo proslediti i više indeksa koji želimo da selektujemo.

In [66]:
```
niz3[[0,2,4,5]]
```

Out[66]:
```
array([100, 100,  22,  13])
```

Sekcija 4 - NumPy funkcionalnosti, NumPy operacije nad nizom

Nad nizovima se mogu vršiti i aritmetičke operacije poput sabiranja, oduzimanja, množenja itd.

Pokažimo neke od navedenih:

In [43]:
```
niz6 = np.arange(0,10)
niz7 = np.arange(0,10)
```

In [44]:
```
niz6 + niz7
```

Out[44]:
```
array([ 0,  2,  4,  6,  8, 10, 12, 14, 16, 18])
```

In [45]:
```
niz6-niz7
```

Out[45]:
```
array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0])
```

In [46]:
```
niz6*niz7
```

Out[46]:
```
array([ 0,  1,  4,  9, 16, 25, 36, 49, 64, 81])
```

In [47]:
```
niz6/niz7
```

```
C:\Users\jolov\AppData\Local\Temp\ipykernel_23144\3610924102.py:1: RuntimeWarning: i
nvalid value encountered in divide
  niz6/niz7
```

Out[47]:
```
array([nan,  1.,  1.,  1.,  1.,  1.,  1.,  1.,  1.,  1.])
```

Primetimo da smo prilikom deljenja sa nulom dobili upozorenje ali ne i grešku.

Postoje i dodatne matematičke operacije poput kvadriranja, kvadratnog korena i slično.

In [50]:
```
niz6**2
```

Out[50]:
```
array([ 0,  1,  4,  9, 16, 25, 36, 49, 64, 81])
```

In [51]:
```
np.sqrt(niz6)
```

Out[51]:
```
array([0.        , 1.        , 1.41421356, 1.73205081, 2.        ,
       2.23606798, 2.44948974, 2.64575131, 2.82842712, 3.        ])
```

In [52]:
```
np.exp(niz6)
```

Out[52]:
```
array([1.00000000e+00, 2.71828183e+00, 7.38905610e+00, 2.00855369e+01,
       5.45981500e+01, 1.48413159e+02, 4.03428793e+02, 1.09663316e+03,
       2.98095799e+03, 8.10308393e+03])
```

In [53]:
```
np.sin(niz6)
```

Out[53]:
```
array([ 0.        ,  0.84147098,  0.90929743,  0.14112001, -0.7568025 ,
       -0.95892427, -0.2794155 ,  0.6569866 ,  0.98935825,  0.41211849])
```

In [54]:
```
np.log(niz6)
```

```
C:\Users\jolov\AppData\Local\Temp\ipykernel_23144\730120308.py:1: RuntimeWarning: di
vide by zero encountered in log
  np.log(niz6)
```

Out[54]:
```
array([      -inf, 0.        , 0.69314718, 1.09861229, 1.38629436,
       1.60943791, 1.79175947, 1.94591015, 2.07944154, 2.19722458])
```

Naravno, možemo i množiti vektor (naš np niz) skalarom.

In [55]:
```
niz6*10
```

Out[55]:
```
array([ 0, 10, 20, 30, 40, 50, 60, 70, 80, 90])
```

Bonus pitanje: Šta bi bio rezultat da običnu listu pomnožimo skalarom? Prirodno možemo i deliti skalarom, sabirati i odizimati skalar od vektora - efekat je da se operacija sa skalarom izvršava nad svakim elementom pojedinačno.

In [59]:
```
print(niz6/10)
print(niz6-10)
print(niz6+100)
```

```
[0.  0.1 0.2 0.3 0.4 0.5 0.6 0.7 0.8 0.9]
[-10  -9  -8  -7  -6  -5  -4  -3  -2  -1]
[100 101 102 103 104 105 106 107 108 109]
```

Ali nam je NumPy funkcijom dot obezbedio vektorsko množenje dva niza uz pomoć funkcije dot.

In [63]:
```
print(niz6.dot(niz7))
#mozemo ga racunati i na sledeci nacin:
```

```
print(np.sum(niz6*niz7))
```

```
285
285
```

Sekcija 5 - NumPy funkcionalnosti, NumPy matrice

NumPy sadrži i implementaciju matrica koje su zapravo samo višedimenzionalni nizovi tipa numpy.ndarray. Biblioteka nam omogućava brojne operacije nad matricama poput množenja i sabiranja kako i sa matricama tako i sa skalarom, inverzija matrice, transponovanje matrice itd.

Definišimo jednu 2x2 matricu:

In [88]:
```
matrica = np.array([[44,29],[18,87]])
print(matrica)
```

```
[[44 29]
 [18 87]]
```

Ispišimo dimenzije matrice preko funkcije shape:

In [89]:
```
matrica.shape
```

Out[89]:
```
(2, 2)
```

Pokažimo razlike između element wise i dot(vektorskog) množenja:

In [91]:
```
print(matrica*matrica)
print("***********")
print(matrica.dot(matrica))
```

```
[[1936  841]
 [ 324 7569]]
***********
[[2458 3799]
 [2358 8091]]
```

Možemo množiti i matricu skalarom ili vektorom bazirano na elementima:

In [92]:
```
vektor = np.array([2,2])
print(matrica*vektor)
print(matrica*10)
```

```
[[ 88  58]
 [ 36 174]]
[[440 290]
 [180 870]]
```

Pazeći na dimenzije možemo računati i vektorski proizvod na sledeći način:

In [93]:
```
matrica.dot(vektor)
```

Out[93]:
```
array([146, 210])
```

Možemo sabirati i oduzimati matrice po elementima:

In [94]:
```
print(matrica+matrica)
print(matrica-matrica)
```

```
[[ 88  58]
 [ 36 174]]
[[0 0]
 [0 0]]
```

Možemo dodavati redove i kolone matrici, pazeći na dimenzije.

In [102…
```
matrica = np.vstack((matrica,np.array([2,2])))
```

In [104…
```
print(matrica)
```

```
[[44 29]
 [18 87]
 [ 2  2]]
```

In [110…
```
matrica = np.hstack((matrica,np.array([[3],[3],[3]])))
```

In [111…
```
print(matrica)
```

```
[[44 29  3]
 [18 87  3]
 [ 2  2  3]]
```

Dakle, funkcijom vstack možemo dodati redove matrici, dok funkcijom hstack dodajemo kolone.

Takođe, vrlo slično nizovima možemo generisati matrice. Pomoću funkcija zeros, ones, random i slično. Pokažimo neke od navedenih:

In [112…
```
print(np.ones((5,5)))
print(np.zeros((5,5)))
print(np.random.random((5,5)))
```

```
[[1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1.]]
[[0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]
 [0. 0. 0. 0. 0.]]
[[0.09132213 0.91131938 0.98217673 0.65814444 0.01713217]
 [0.50524135 0.57930354 0.84783602 0.3942966  0.21346572]
 [0.33276609 0.13160841 0.39891455 0.30611872 0.97233821]
 [0.93314978 0.77244291 0.53643559 0.7572875  0.28098299]
 [0.24688937 0.38143549 0.36044882 0.00579858 0.89873924]]
```

Pokažimo još da slajseri rade veoma slično kao i kod niza, uz činjenicu da imamo još jednu dimenziju. Izbor cele matrice slajserom možemo uraditi:

In [127…
```
matrica[:,:] # ili samo matrica[:]
```

Out[127…
```
array([[44, 29,  3],
       [18, 87,  3],
       [ 2,  2,  3]])
```

Recimo da želimo da izaberemo samo jedan red - treći.

In [129…
```
matrica[2,:]
```

Out[129…
```
array([2, 2, 3])
```

Ili samo treću kolonu...

In [130…
```
matrica[:,2]
```

Out[130…
```
array([3, 3, 3])
```

Možemo želeti da izaberemo tačno određen element, recimo element druge kolone i drugog reda:

In [132…
```
matrica[1,1]
```

Out[132…
```
87
```

Ili podmatricu koju čine elementi iz prvog i drugog reda i kolone.

In [137…
```
matrica[0:2,0:2]
```

Out[137…
```
array([[44, 29],
       [18, 87]])
```

Recimo da želimo da izaberemo elemente po nekom uslovu, recimo da su veći od 10:

In [140…
```
matrica[matrica>10]
```

Out[140…
```
array([44, 29, 18, 87])
```

Zadaci za vežbu:

- kreirati np niz od 10 nula
- kreirati niz celobrojnih vrednosti od 10 do 50
- kreirati 3x3 matricu sa vrednostima od 0 do 8
- iskorititi numpy biblioteku za generisanje slučajnog broja između 0 i 1
- iskoristiti numpy za generisanje niza od 25 slučajnih brojeva uzetih iz normalne raspodele
- Kreirati matricu dimenzija 5x5 sa brojevima od 1 do 25, uz pomoć funkcija reshape i arange5izračunati njenu sumu, vektorski prozvod sa identičnom matricom, naći standardnu devijaciju svih vrednosti u matrici, naći sumu svih kolona u matrici.]])

Mala pomoć za zadatke je u objašnjenju još jedne funkcije - reshape:

In [146…
```
matrica = np.array([12,12,12,12,12,12])
matrica = np.reshape(matrica,(2,3))
matrica
```

Out[146…
```
array([[12, 12, 12],
       [12, 12, 12]])
```

In [ ]:
```
```